Tecnologia em Análise e Desenvolvimento de Sistemas

INSTITUTO FEDERAL
São Paulo
Câmpus São Carlos

APR1 – Algoritmos e Programação

Manipulação de Arquivos

Profa. Dra. Eloize Seno

INSTITUTO FEDERAL
São Paulo
Câmpus São Carlos

Arquivos

- Os dados que foram usados nos programas até agora OU
  - estão inseridos diretamente no código OU
  - são digitados pelo usuário a medida que são lidos

- Na vida real dados estão em ARQUIVOS

- Arquivos ficam armazenados no disco rígido de um computador

- Devemos sempre **abrir** (*open*) arquivos antes de usá-los e **fechar** (*close*) os arquivos depois de que tivermos terminado de utilizá-los

Arquivos

- Métodos para abrir e fechar um arquivo

Nome de Método	Uso	Explicação
`open`	`open(nome_arquivo,'r')`	Abre um arquivo chamado nome_arquivo e o usa para leitura. Retorna uma referëncia para um objeto *file*.
`open`	`open(nome_arquivo,'w')`	Abre um arquivo chamado nome_arquivo e o usa para escrita. Retorna uma referëncia para um objeto *file*.
`close`	`ref_arquivo.close()`	Utilização do arquivo referenciado pela variável ref_arquivo terminou.

Arquivos

- Depois de aberto um arquivo passa a ser um objeto Python de maneira semelhante a outros dados

- A maneira que os arquivos são localizados no disco é através do seu **caminho** (*path*), cujo formato depende do sistema operacional utilizado
  - import os

- se o seu arquivo de dados e o seu programa Python estão no mesmo diretório, você pode usar simplesmente o nome do **arquivo. open(‘arquivo_teste.txt','r')**

Existência de arquivo

- A função a seguir testa se um arquivo existe antes de tentar abri-lo:

```
def Existe_Arquivo(nome):
    import os
    if os.path.exists(nome):
        return True
    else:
        return False
```

Iterando sobre as linhas de um Arquivo

- Como arquivos de texto são uma sequência de linhas com texto, deve-se usar um **laço de repetição** para iterar sobre cada linha do arquivo

- Um linha (*line*) de um arquivo é definida como uma sequência de caracteres e inclui um caractere especial chamado de nova linha (‘\n’)

- Quando estamos digitando um programa em Python e pressionamos a tecla **enter**, o editor insere o caractere de nova linha no texto naquele ponto

Iterando sobre as linhas de um Arquivo com for

- A medida com que o laço ***for*** itera sobre cada linha do arquivo, a variável de controle do laço conterá uma referência para uma **string**, que contém o conteúdo da linha corrente do arquivo
- O padrão geral para processar cada linha de um arquivo texto é o seguinte:

```
for linha in ref_arquivo:
    comando1
    comando2
    ...
```

Iterando sobre as linhas de um Arquivo

- Exemplo:

```
ref_arq=open("cad_alunos.txt",'r')
 for linha in ref_arq:
        aluno = linha.split()
        prontuario=str(aluno[0])
        nome=str(aluno[1])
        email=str(aluno[2])
 ref_arq.close()
```

Métodos leitura e escrita em arquivos

Método	Uso	Efeito
`write`	`ref_arquivo.write(s)`	Adiciona o string `s` no final do arquivo. ref_arquivo deve ser uma referência a um arquivo que foi aberto para escrita (`"w"`).
`read(n)`	`ref_arquivo.read()`	Lê e retorna um string de `n` caracteres ou o arquivo inteiro como um string se `n` não é fornecido.
`readline(n)`	`ref_arquivo.readline()`	Retorna a próxima linha do arquivo com todo o texto e incluindo o caractere de nova linha. Se `n` é fornecido como argumento então somente n caracteres são retornados se a linha tem mais do que `n` caracteres.
`readlines(n)`	`ref_arquivo.readlines()`	Retorna uma lista de strings, cada um representado o conteúdo de uma linha do arquivo. Se `n` não é fornecido, todas as linhas do arquivo são retornadas. Se `n` é fornecido como argumento então `n` caracteres serão lidos mas `n` é arrendondado para cima de tal forma que uma linha inteira seja retornada.

Métodos de leitura em arquivos

- O método **readline** lê uma linha de um arquivo e retorna essa linha como uma **string**
  - A string retornada por readline conterá o caractere de nova linha ('\n') no final
  - Este método retorna a string vazia ("") quando chega ao final do arquivo
- O método **readlines** retorna o todo o conteúdo do arquivo em uma lista de strings, cada item da lista representa uma linha do arquivo
- Também é possível ler todo o conteúdo de um arquivo em uma **única string** como o método **read**

Métodos de leitura em arquivos

- Utilizando o laço while:

```
ref_arquivo = open(“cad_alunos.txt","r")
print('Dados dos Alunos:')
linha = ref_arquivo.readline()
while linha:
aluno = linha.split()
    chave=str(aluno[0])
    nome=str(aluno[1])
    email=str(aluno[2])
ref_arquivo.close()
```

Atividade prática

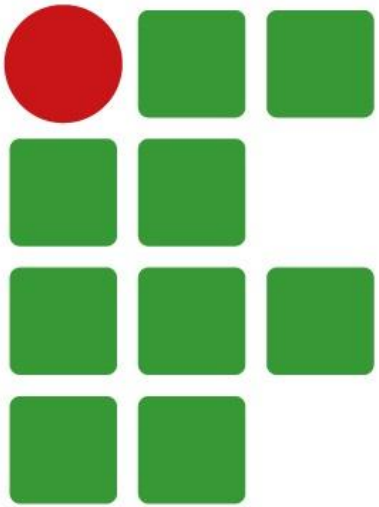
Crie um arquivo denominado “estudantes.txt” que contém uma linha para cada aluno de uma turma de estudantes. O nome de cada estudante está no início da cada linha e é seguido pelas suas notas. Com esse arquivo faça:

```
jose 10 15 20 30 40
pedro 23 16 19 22
suzana 8 22 17 14 32 17 24 21 2 9 11 17
gisela 12 28 21 45 26 10
joao 14 32 25 16 89
```

a) Uma função que retorna o nome do estudante que possui mais de 6 notas;

b) Uma função que calcula a média das notas e imprime o nome e a média de cada estudante;

c) Uma função que calcula a nota mínima e máxima de cada estudante e imprima o nome de cada aluno junto com a sua maior e a menor nota.



INSTITUTO FEDERAL
São Paulo
Câmpus São Carlos